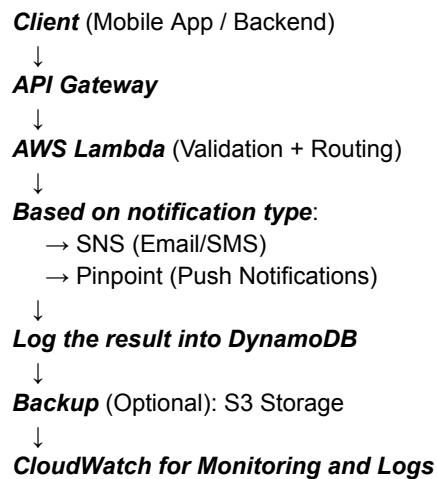


Services used in this Project

<u>AWS Service</u>	<u>Purpose</u>
API Gateway	Public HTTP endpoint to receive notification requests
AWS Lambda	Process requests and route to correct notification channels
Amazon SNS	Send Email and SMS messages
Amazon Pinpoint	Send Push Notifications
DynamoDB	Store notification logs and statuses
AWS KMS	Encrypt sensitive data (like email addresses)
AWS CloudWatch	Monitor Lambda performance, API traffic, and generate alarms
AWS S3	Store backup notification logs (optional)
IAM Roles and Policies	Secure permissions for Lambda, API Gateway, and other services

Streamline of action between Services



Reasons Why It's a Strong Architecture

Strength	Reason
Serverless and Event-Driven	Best fit for notifications workload
Cost-Efficient	Pay only per execution/request
Highly Scalable	Lambda and API Gateway auto-scale
Secure	Encrypted, least privilege access
Extensible	Easy to add new channels (e.g., WhatsApp, in-app notifications)
IaC	Full reproducibility with Terraform

High-Level System Architecture Overview

In this project, I'm using **Terraform** to provision and manage AWS infrastructure for a scalable and event-driven **Notification Platform**. The use of Terraform allows for a **clean, modular, and reusable infrastructure-as-code** setup, making deployments easier to manage, version, and troubleshoot.

I provisioned the **REST API Gateway**, the **Lambda service**, and supporting components such as **SNS**, **Pinpoint**, **DynamoDB**, and **CloudWatch**. These services work together to receive, process, route, log, and monitor notification requests.

I chose **AWS Lambda over Fargate** because Lambda is better suited for **event-driven, short-lived notification tasks** and doesn't require containerization. This choice offers **lower cost, automatic scaling**, and a **simplified serverless architecture** that's ideal for handling sporadic notification events with minimal overhead.

This serverless notification platform is built for scalability, cost-efficiency, and ease of maintenance using AWS services provisioned with Terraform. The core workflow begins when a

client sends an HTTP POST request to an **API Gateway** endpoint, which triggers an **AWS Lambda function**. This function parses the request, identifies the notification type (email, SMS, or push), and routes it to the appropriate service.

- **Amazon SNS** handles email and SMS delivery.
- **Amazon Pinpoint**, or third-party tools like **Firebase** or **OneSignal**, are used for push notifications.
- Logs are stored in **DynamoDB** for traceability.
- Metrics and logs are pushed to **CloudWatch**, with optional backups to **S3**.

The system is fully scalable. **Lambda**, **API Gateway**, and **DynamoDB** auto-scale to handle traffic spikes, and all resources span multiple **Availability Zones** for high availability.

Security is enforced using **least-privilege IAM roles**, **KMS encryption** for data at rest, and **HTTPS** for secure communication. **Cognito** or IAM-based authentication can secure the API layer.

Monitoring is handled by **CloudWatch Logs and Metrics**, with **alarms** for key metrics and optional use of **X-Ray** for deeper traceability.

The Lambda function includes a routing layer that makes the system **channel-agnostic**, and adding new channels (like Slack or WhatsApp) is as simple as extending the Lambda logic and storing secrets in **AWS Secrets Manager**. This makes the platform easy to test, extend, and adapt to new communication needs.